SeaBird ISA

Reference Manual

Hybrid CISC/RISC Architecture

Technical Specification

makateotakanawa@gmail.com
Research and Development at Meisei

December 7, 2025
UPDATED: January 28, 2026

Document Version: 1.1
Classification: Technical Documentation

Meisei - Naoki Saito
For Use with Meisei General-Purpose Processors

Document Information

Field	Value
Document Title	SeaBird ISA Reference Manual
Version	1.1
Date	December 7, 2025 — UPDATED: January 28, 2026
Status	1.1
Architecture	Hybrid CISC/RISC
Modes Supported	Clownfish (16-bit), Tetra (32-bit), Dragonet (64-bit), Droplet (128-bit)
Instruction Count	160 instructions
Register Count	32 general-purpose registers (R0–R31)

Abstract

The SeaBird ISA architecture is a modern hybrid CISC/RISC architecture intended for use with high-performance designs. This architecture blends visible CISC machine-level instructions with a more internal execution model for micro-operations similar to a conventional RISC architecture.

The architecture supports a wide array of operating modes (either 16-bit, 32-bit, or 64-bit operation), full vectorization capabilities, and remains compatible with RISC-V.

This manual constitutes a full specification of the instruction set, encodings, register architecture, memory model, and implementation techniques for use by compiler writers, operating system designers, emulator implementors, and hardware engineers.

Contents

1 Introduction 6

2 Operating Modes and Memory Model 6
- 2.1 Operating Modes . . . 6
- 2.2 Memory Model . . . 7
  - 2.2.1 Mode-Independent Instruction Set . . . 7
  - 2.2.2 Privilege Enforcement via MMU . . . 7
  - 2.2.3 Mode Privilege Summary . . . 8

3 Registers 8

3.1 General-Purpose Registers (GPRs) . . . 9
3.1.1 Primary GPRs (R0–R15) . . . 9
3.1.2 Extended General Registers (R16–R31) . . . 9
3.2 Register Access Rules . . . 10
3.2.1 Dual Naming . . . 10
3.2.2 Width Rules Across Operating Modes . . . 10
3.2.3 Subregister Views . . . 10
3.3 Special Registers . . . 11

4 Instruction Syntax 11
4.1 Core Syntax Rules . . . 11
4.2 Syntax Examples . . . 12

5 Addressing Modes 12
5.1 Addressing Mode Examples . . . 13
5.2 Micro-Op Decomposition . . . 14

6 Instruction Formats 14
6.1 CISC Frontend Formats . . . 14
6.2 Detailed Format Descriptions . . . 14
6.2.1 R-Type (Register Only) . . . 14
6.2.2 I-Type (Immediate) . . . 15
6.2.3 M-Type (Memory Access) . . . 15
6.2.4 B-Type (Branch / Jump) . . . 15
6.2.5 S-Type (System / Control) . . . 15
6.2.6 V-Type (Vector / Matrix) . . . 16
6.2.7 X-Type (Extended / Microcoded) . . . 16

7 Instruction Encoding 16
7.1 Binary Encoding Overview . . . 16
7.2 Prefix Byte (Optional, 1 byte) . . . 17
7.3 Opcode Bytes (1–3 bytes) . . . 17
7.4 ModR/M Byte (Optional) . . . 18
7.5 SIB Byte (Optional) . . . 18
7.6 Displacement / Immediate (0, 1, 4, or 8 bytes) . . . 18
7.7 Addressing Modes Summary . . . 19
7.8 Encoding Examples . . . 19
7.8.1 Example 1: Simple Arithmetic . . . 19
7.8.2 Example 2: Complex Addressing . . . 19
7.8.3 Example 3: Vector Operation . . . 20
7.8.4 Example 4: Immediate Load . . . 20
7.9 Encoding Legend . . . 20

8 Instruction Encoding & Micro-Op Mapping 21
8.1 Data Movement (24 instructions) 21
8.2 Arithmetic (26 instructions) 22
8.3 Logic & Bit Operations (18 instructions) 23
8.4 Comparison (10 instructions) 23
8.5 Branch / Control Flow (20 instructions) 24
8.6 Memory / Addressing Extensions (12 instructions) 25
8.7 System & Privileged (15 instructions) 25
8.8 Stack & Call Frame (10 instructions) 26
8.9 SIMD / Vector (15 instructions) 27
8.10 Floating Point (10 instructions) 27

9 Micro-Operations Architecture 28
9.1 Hybrid CISC/RISC Design Philosophy 28
9.2 Micro-Op Decomposition Rules 29
9.2.1 Rule 1: Single-Operation Instructions → 1 µop 29
9.2.2 Rule 2: Memory Operations → 2–3 µops 29
9.2.3 Rule 3: Complex Addressing Modes → 3–5 µops 29
9.2.4 Rule 4: Arithmetic with Immediate → 1–2 µops 30
9.2.5 Rule 5: CISC-Style Compound Operations → 2–6 µops 30
9.2.6 Rule 6: Branches → 1 µop 31
9.2.7 Rule 7: Vector Operations → N µops 31
9.2.8 Rule 8: Microcoded Instructions → Variable µops 31
9.3 Micro-Op Execution Properties 31
9.3.1 Micro-Op Characteristics 31
9.3.2 Backend Optimization Opportunities 32
9.3.3 Example: Full Instruction Pipeline 32

10 Instruction Set Reference 32
10.1 Data Movement (24 instructions) 33
10.1.1 Basic Moves 33
10.1.2 Memory Access 33
10.1.3 Special Variants 33
10.2 Arithmetic (26 instructions) 33
10.2.1 Integer Arithmetic 33
10.2.2 Extended / 64-bit 34
10.2.3 Saturating Arithmetic 34
10.2.4 Bit Arithmetic 34
10.3 Logic & Bit Operations (18 instructions) 34
10.4 Comparison (10 instructions) 35
10.5 Branching & Control Flow (20 instructions) 35
10.5.1 Unconditional 35
10.5.2 Conditional (flag-based) 35

10.5.3 Register-based . . . 36
10.5.4 Special . . . 36
10.6 Memory / Addressing Extensions (12 instructions) . . . 36
10.7 System & Privileged (15 instructions) . . . 36
10.8 Stack & Call Frame (10 instructions) . . . 37
10.9 SIMD / Vector (15 instructions) . . . 37
10.10 Floating Point (10 instructions) . . . 38

11 Assembler Directives 38
11.1 Code & Data Layout Directives . . . 38
11.2 Symbol & Label Directives . . . 38
11.3 Data Definition Directives . . . 39
11.4 Control Directives . . . 39
11.5 Architecture, Mode & Optimization Control . . . 39
11.6 Debugging, Listing & Metadata . . . 40
11.7 Special Optimization Directives . . . 40

12 Example Instructions With Micro-Op Breakdown 41

13 Instruction Latency & Performance Characteristics 41
13.1 Typical Instruction Latencies . . . 41
13.2 Cache and Memory Hierarchy . . . 41
13.3 Pipeline Characteristics . . . 42

14 FLAGS Register 42
14.1 Status Flags . . . 42
14.2 Flag Modification . . . 43

15 Exception & Interrupt Handling 43
15.1 Exception Vector Table . . . 43
15.2 Exception Handling Mechanism . . . 44
15.3 Privilege Level Transitions . . . 44

16 Control Registers 45
16.1 System Control Registers . . . 45
16.2 CR0 Bit Definitions . . . 45

17 Memory Management 45
17.1 Paging Mechanism . . . 45
17.2 Page Table Entry Format . . . 46
17.3 Translation Lookaside Buffer (TLB) . . . 46

18 Programming Examples 47
18.1 Example 1: Simple Loop . . . 47

18.2 Example 2: Function Call with Stack Frame 47
18.3 Example 3: Vector Operations 48
18.4 Example 4: System Call 48

19 Implementation Notes 48
19.1 Design Considerations for Implementers 48
19.1.1 Frontend Decoder 48
19.1.2 Execution Backend 49
19.1.3 Branch Prediction 49
19.2 Compliance Levels 49

20 Revision History 49

21 Acknowledgments 50

22 Glossary 50

1 Introduction

The SeaBird ISA is a hybrid CISC/RISC architecture designed to:

- Allow highly expressive instructions for compact, human-readable assembly.
- Break CISC instructions into simple RISC-like micro-ops internally for high performance.
- Support multiple operating modes:
  - Clownfish (16-bit real mode)
  - Tetra (32-bit protected mode)
  - Dragonet (64-bit long mode)
  - Copycat (RISC-V compatibility mode)
- Support vector and floating-point operations in addition to scalar operations.
- Provide a flexible register naming scheme that allows both Intel-style semantic names and Rx numeric aliases.

The architecture is designed for modern CPU pipelines, allowing micro-op fusion, vectorization, and efficient memory addressing.

2 Operating Modes and Memory Model

2.1 Operating Modes

Mode	Bit Width	Purpose	Notes
Clownfish	16-bit	Legacy/boot real-mode	IP=16-bit, Registers=16-bit
Tetra	32-bit	Protected mode	Supports 32-bit virtual memory, standard privilege rings
Dragonet	64-bit	Long mode	64-bit registers, supports 64-bit addressing, RIP-relative addressing
Copycat	32/64-bit	Compatibility mode	Maps R0–R31 to standard RISC-V general-purpose registers
Droplet	128-bit	Hypervisor mode	Virtual wide precision mode for complex computing tasks

Vector Reset: 0xCAFE (entry point for vector tables)

2.2 Memory Model

- Flat linear memory in Tetra/Dragonet modes
- Memory protection and masking handled by the memory controller / MMU
- Instructions are independent of mode; access faults occur via MMU if privileges are violated

2.2.1 Mode-Independent Instruction Set

Critical Design Principle: All instructions in the SeaBird ISA are mode-independent. The instruction encoding, mnemonics, and behavior remain identical across all privilege levels and operating modes.

Examples:

```
LD  R1, [R2]              ; loads from memory (all modes)
ST  [R3], R4              ; stores to memory (all modes)
MOV R5, R6                ; register move (all modes)
ADD R7, R8, R9            ; arithmetic (all modes)
JMP label                 ; jump (all modes)
```

Nothing about the instruction encoding changes. Nothing about the mnemonic changes. The instruction simply generates a memory request or operation.

2.2.2 Privilege Enforcement via MMU

The Memory Management Unit (MMU) / memory controller handles all permission checks *after* instruction decode:

Process Flow:

1. Instruction decoded (e.g., LD R1, [0xFFFF0000])
2. Memory request issued to MMU
3. MMU checks:
   - Current privilege level (user/kernel/hypervisor)
   - Address mapping validity
   - Access permissions (read/write/execute)
   - Protection attributes
4. MMU response:
   - **Success:** Return data / complete operation
   - **Failure:** Trigger exception (access fault, page fault, privilege violation, segmentation fault)

Example Scenarios:

Mode	Instruction	Behavior
User	LD R1, [0xFFFF0000]	**Fault** if kernel-only address
User	ST [user_data], R2	**Success** if mapped & writable
Kernel	LD R1, [0xFFFF0000]	**Success** (kernel memory accessible)
Kernel	ST [io_port], R3	**Success** (I/O access allowed)
Hypervisor	LD R1, [guest_mem]	**Success** (VM memory accessible)

2.2.3 Mode Privilege Summary

User Mode:

- Can execute all instructions (LD, ST, ADD, JMP, etc.)
- *Cannot* access kernel memory, I/O ranges, page tables, or privileged control registers
- Memory violations trigger exceptions handled by kernel

Kernel Mode:

- Same instruction set as user mode
- *Can* access privileged memory regions, I/O ports, and control registers
- Full access to system resources

Hypervisor Mode (optional):

- Same instruction set as user/kernel modes
- *Can* access VM guest memory mappings and virtualization control structures
- Highest privilege level

Design Advantage: One unified instruction set across all privilege levels. Mode determines *whether access succeeds*, not *what instructions are available*.

3 Registers

The SeaBird ISA provides a comprehensive register set with dual naming conventions, supporting both modern numeric naming (R0–R31) and legacy-style semantic aliases (AX, BX, etc.). Register width varies by operating mode.

3.1 General-Purpose Registers (GPRs)

3.1.1 Primary GPRs (R0–R15)

Available in all modes (Clownfish, Tetra, Dragonet, RISC-V):

Physical	SeaBird Name	Legacy Alias	Purpose Hint
R0	A0	AX	Primary accumulator
R1	A1	BX	Base/index utility
R2	A2	CX	Counter register
R3	A3	DX	Data register
R4	B0	SI	Source pointer / string ops
R5	B1	DI	Destination pointer
R6	B2	BP	Base pointer
R7	B3	SP	Stack pointer
R8	C0	R8W/R8D/R8	Extra GPR
R9	C1	R9	Extra GPR
R10	C2	R10	Extra GPR
R11	C3	R11	Extra GPR
R12	D0	R12	Extra GPR
R13	D1	R13	Extra GPR
R14	D2	R14	Extra GPR
R15	D3	R15	Extra GPR

3.1.2 Extended General Registers (R16–R31)

Available in Dragonet (64-bit) mode only:

Physical	SeaBird Name	Purpose
R16	E0	Environment/exception scratch
R17	E1	Syscall argument 0
R18	E2	Syscall argument 1
R19	E3	Syscall argument 2
R20	F0	FP scratch 0
R21	F1	FP scratch 1
R22	F2	FP scratch 2
R23	F3	FP scratch 3
R24	G0	General purpose
R25	G1	General purpose
R26	G2	General purpose
R27	G3	General purpose
R28	T0	Temporary (µop engine)
R29	T1	Temporary (µop engine)
R30	T2	Temporary (µop engine)
R31	T3	Temporary (µop engine)

Note: The T-class registers (R28–R31) are ideal for micro-op expansion and addressing-mode calculations, serving as internal scratch registers for the µop execution engine.

3.2 Register Access Rules

3.2.1 Dual Naming

All registers are accessible via multiple aliases:

- **R0 = A0 = AX** (all refer to the same physical register)
- **R3 = A3 = DX**
- **R5 = B1 = DI**
- **R7 = B3 = SP**

3.2.2 Width Rules Across Operating Modes

Mode	Register Width	Pointer Width
Clownfish	16-bit	16-bit
Tetra	32-bit	32-bit
Dragonet	64-bit	64-bit
Copycat	Uses R0–R31 as RV x0–x31	32/64-bit

3.2.3 Subregister Views

Similar to x86-64, registers support partial access:

Suffix	Meaning
A0B	Lower 8 bits (byte)
A0W	Lower 16 bits (word)
A0D	Lower 32 bits (dword)
A0	Full 64 bits (qword, Dragonet only)

Examples:

```
MOV A0W, #0x1234      ; move to lower 16 bits
MOV A0D, #0xDEADBEEF  ; move to lower 32 bits
MOV A0, #0xCAFEBABE   ; move to full 64-bit register
```

In Clownfish mode (16-bit):

- AX refers to the full 16-bit view of R0

In Tetra mode (32-bit):

- A0D or AX refers to the full 32-bit view of R0

In Dragonet mode (64-bit):

- A0 refers to the full 64-bit view of R0

3.3 Special Registers

These registers cannot be accessed as Rn and require special instructions (similar to x86 CR/DR registers):

Name	Purpose
IP	Instruction pointer (16/32/64-bit depending on mode)
FLAGS	Status flags (zero, carry, overflow, sign, etc.)
CFAR	CISC Frontend Address Register (RIP + prefix info)
UCF	Micro-op control flags
VECCTL	Vector mode control
SPTR	Shadow stack pointer (optional security feature)

Access: Special registers are manipulated via dedicated instructions such as RDCR/WRCR or implicit behavior during control flow operations.

4 Instruction Syntax

4.1 Core Syntax Rules

```
MNEMONIC  operand1, operand2, operand3
```

General Syntax Rules:

- **Mnemonics** are case-insensitive (ADD, add, AdD are identical)
- **Operands** are separated by commas
- **Registers** begin with R:
  - R0–R15 (Clownfish mode)
  - R0–R31 (Tetra, Dragonet, RISC-V modes)
  - Semantic names also supported (AX, BX, SI, DI, etc.)

- **Immediate values:**
  - Decimal: 42
  - Hex: 0x2A, 0xFF
  - Binary: 0b101010, 0b1010
  - Character: 'A'
- **Comments:** ; this is a comment
- **Labels** end with a colon: loop_start:
- **Directives** begin with . (see Assembler Directives section)
- **Whitespace** is ignored except inside strings

4.2 Syntax Examples

```
; Case-insensitive mnemonics
ADD R1, R2, R3
add r1, r2, r3        ; equivalent to above

; Immediate values
MOVI R0, #42          ; decimal
MOVI R1, #0x2A        ; hexadecimal
MOVI R2, #0b101010    ; binary
MOVI R3, #'A'         ; character literal

; Labels and jumps
loop_start:
    ADD R1, R2, R3
    DEC R4
    JNZ loop_start    ; jump if not zero

; Comments
MOV R0, R1            ; copy R1 to R0
```

5 Addressing Modes

The SeaBird ISA uses CISC-style addressing modes in the programmer-visible frontend, which are internally decomposed into RISC-like micro-ops by the backend execution engine. This unified addressing model is available across all operating modes.

Mode	Syntax	Description
Register Direct	R1	Operand in register
Immediate	#42, #0xFF	Constant value (decimal, hex, binary)
Register Indirect	[R2]	Memory pointed by register
Register + Offset	[R2+16], [R7+0x20]	Base + immediate displacement
Base + Index (Scaled)	[R1+R3*4], [R2+R6*8]	Base + scaled index (typical array access)
Base + Index + Displacement	[R4+R5*2+32]	Full complex addressing
PC-relative	label, label(RIP)	PC-relative (Dragonet mode only)
Absolute Address	[0x9000]	Direct memory address (primarily Clownfish/Tetra)

5.1 Addressing Mode Examples

```
; Register Direct
MOV R1, R2                ; copy R2 to R1

; Immediate
MOVI R3, #42              ; load immediate value 42
ADDI R4, #0xFF            ; add 255 to R4

; Register Indirect
LD R5, [R6]               ; load from address in R6
ST [R7], R8               ; store R8 to address in R7

; Register + Offset
LD R1, [R2+16]            ; load from R2 + 16
ST [R4+0x20], R3          ; store to R4 + 32

; Base + Index (Scaled)
LD R0, [R1+R3*4]          ; array access: base + index*4
ST [R2+R6*8], R5          ; store with 8-byte stride

; Base + Index + Displacement
LD R7, [R4+R5*2+32]       ; complex: base + index*2 + 32

; PC-relative (Dragonet mode)
JMP forward               ; relative to current IP
CALL function(RIP)        ; RIP-relative call

; Absolute Address (Clownfish/Tetra)
LD R1, [0x9000]           ; load from fixed address
ST [0xFFFF], R2           ; store to fixed address

forward:
    RET
```

5.2 Micro-Op Decomposition

Complex addressing modes are transparently broken into micro-ops:

```
; Programmer writes:
LD R1, [R2+R3*4+32]

; Internal micro-op decomposition:
uMUL  Rtmp, R3, #4        ; scale index
uADD  Rtmp, R2, Rtmp      ; add base
uADD  Rtmp, Rtmp, #32     ; add displacement
uLOAD R1, [Rtmp]          ; perform load
```

6 Instruction Formats

The SeaBird ISA presents compact, expressive CISC-style instruction formats to the programmer. Internally, the backend decomposes these into RISC-like micro-ops during execution.

6.1 CISC Frontend Formats

Format	Syntax	Usage
R-type	MNEMONIC Rdst, Rsrc1, Rsrc2	Register-only operations (arithmetic, logic, compare, bitwise)
I-type	MNEMONIC Rdst, Rsrc, #imm	Immediate arithmetic / logic
M-type	MNEMONIC Rdst, [mem] MNEMONIC [mem], Rsrc	Memory access (load/store)
B-type	BCOND label JMP label / JZ label	Branches and jumps (conditional/unconditional)
S-type	MNEMONIC sysop	System / control instructions (interrupts, mode-switch, flags)
V-type	VADD V0, V1, V2 VMUL V3, V1, #5	Vector/matrix operations
X-type	MNEMONIC extended	Extended / microcoded instructions (multi-µop guaranteed expansion)

6.2 Detailed Format Descriptions

6.2.1 R-Type (Register Only)

Register-to-register operations for arithmetic, logic, comparison, and bitwise operations.

```
ADD   R1, R2, R3          ; R1 = R2 + R3
SUB   R4, R5, R6          ; R4 = R5 - R6
AND   R7, R8, R9          ; R7 = R8 & R9
CMP   R1, R2              ; compare R1 and R2
MAX   R3, R4              ; R3 = max(R3, R4)
```

6.2.2 I-Type (Immediate)

Operations involving immediate (constant) values.

```
ADDI  R1, R2, #42         ; R1 = R2 + 42
MOVI  R3, #0xFF           ; R3 = 255
CMPI  R4, #100            ; compare R4 with 100
SUBI  R5, #8              ; R5 = R5 - 8
```

6.2.3 M-Type (Memory Access)

Load and store operations with various addressing modes.

```
LOAD  R1, [R2]            ; load from [R2]
LOAD  R1, [R2+4]          ; load from [R2+4]
LOAD  R1, [R2+R3*4]       ; load from [R2+R3*4]
STORE [R4], R5            ; store R5 to [R4]
STORE [R4+R5*2+32], R6    ; complex addressing
```

6.2.4 B-Type (Branch / Jump)

Control flow operations, both conditional and unconditional.

```
JMP   label               ; unconditional jump
JZ    loop_start          ; jump if zero
JNZ   exit                ; jump if not zero
JG    greater             ; jump if greater
CALL  function            ; call subroutine
RET                       ; return from subroutine
```

6.2.5 S-Type (System / Control)

System-level operations for interrupts, mode switching, and privileged operations.

```
SYSCALL                   ; system call
HLT                       ; halt processor
CLI                       ; clear interrupts
STI                       ; set interrupts
```

```
RDCR  R1, CR0             ; read control register
WRCR  CR2, R3             ; write control register
```

6.2.6 V-Type (Vector / Matrix)

SIMD and vector operations operating on vector registers.

```
VADD  V0, V1, V2          ; vector add
VMUL  V3, V4, V5          ; vector multiply
VSUB  V6, V7, V8          ; vector subtract
VDIV  V0, V1, V2          ; vector divide
VADD  V3, V1, #5          ; vector-scalar add
```

6.2.7 X-Type (Extended / Microcoded)

Special instructions guaranteed to expand into multiple micro-ops. Used for complex operations.

```
PUSHA                     ; push all GPRs (microcoded)
POPA                      ; pop all GPRs (microcoded)
ENTER #16                 ; create stack frame
LEAVE                     ; destroy stack frame
CPYB  R1, R2, #256        ; copy 256 bytes (microcoded loop)
```

7 Instruction Encoding

7.1 Binary Encoding Overview

The SeaBird ISA uses an x86-style modular binary encoding format. All instructions follow this layout:

`[Prefix][Opcode][ModR/M][SIB][Displacement/Immediate]`

Each instruction may consist of the following components:

1. **Prefix byte** (optional, 0–1 bytes) - Mode, size overrides, vector selectors
2. **Opcode byte(s)** (1–3 bytes) - Instruction identifier
3. **ModR/M byte** (optional) - Register/memory operand encoding
4. **SIB byte** (optional) - Scaled index addressing
5. **Immediate / Displacement** (0, 1, 4, or 8 bytes) - Constants and offsets

This encoding scheme is:

- **CISC-friendly:** Compact representation for programmers
- **Mode-independent:** Same encoding works across 16/32/64-bit modes
- **Extensible:** Easy to add vector and system instructions
- **RISC-backend compatible:** Decomposes cleanly into micro-ops

7.2 Prefix Byte (Optional, 1 byte)

The prefix byte modifies instruction behavior and is only required for overrides or special operations.

Bits	Purpose	Notes
7-6	Mode	00=Clownfish (16-bit), 01=Tetra (32-bit), 10=Dragonet (64-bit), 11=Reserved
5	Address size override	Switch between default mode and 16/32/64-bit addresses
4	Operand size override	Switch default operand size (16/32/64-bit)
3-0	Vector/SIMD selector	Lane width, matrix/vector size, etc.

Usage: If instruction operates in default mode, prefix may be omitted. Required only for overrides, vector operations, or cross-mode compatibility.

7.3 Opcode Bytes (1–3 bytes)

Opcodes are organized by functional class for efficient decoding:

Class	Opcode Range	Count	Bytes
Data movement	0x00–0x1F	24 instructions	1–2
Arithmetic	0x20–0x3F	26 instructions	1
Logic/Bit	0x40–0x51	18 instructions	1
Comparison	0x52–0x5B	10 instructions	1
Branch/Control	0x5C–0x71	20 instructions	1–2
Memory extensions	0x72–0x7D	12 instructions	2
System/Privileged	0x7E–0x8C	15 instructions	2–3
Stack/Call	0x8D–0x96	10 instructions	1–2
SIMD/Vector	0x97–0xA5	15 instructions	2
Floating Point	0xA6–0xAF	10 instructions	2

Encoding Strategy:

- 1 byte: Basic arithmetic, logic, comparison, stack operations
- 2 bytes: Memory operations, vector operations
- 3 bytes: System, privileged, complex CISC instructions (e.g., LEAS, PUSHA/POPA)

7.4 ModR/M Byte (Optional)

The ModR/M byte encodes register and memory operands, similar to x86 but simplified.

Bits	Meaning
7-6	mod (addressing mode)
5-3	reg (register operand or opcode extension)
2-0	r/m (register/memory operand)

mod Field Values:

mod	Addressing Mode
00	[Rbase] - register indirect
01	[Rbase + imm8] - base + 8-bit displacement
10	[Rbase + imm32] - base + 32-bit displacement
11	Register direct (no memory access)

Note: This encoding is cleaner than x86 with no legacy mode complications. It fits all 16/32/64-bit modes uniformly.

7.5 SIB Byte (Optional)

The SIB (Scale-Index-Base) byte enables scaled index addressing for array and structure access.

Bits	Meaning
7-6	scale (00=1, 01=2, 10=4, 11=8)
5-3	index register (R0–R7 in 3 bits, extended via prefix)
2-0	base register (R0–R7 in 3 bits, extended via prefix)

Usage: Only present when instruction uses [base + index*scale + disp] addressing.

7.6 Displacement / Immediate (0, 1, 4, or 8 bytes)

Variable-length field for constants and memory offsets.

Size	Usage	Example
0 bytes	No immediate/displacement	ADD R1, R2, R3
1 byte	Small displacement/immediate	LD R1, [R2+16] (Clownfish)
4 bytes	32-bit displacement/immediate	ADDI R3, #0x12345678 (Tetra)
8 bytes	64-bit displacement/immediate	MOVI R4, #0xCAFEBABE (Dragonet)

7.7 Addressing Modes Summary

Mode	Syntax	Description
Register direct	Rdst	Simple register to register
Memory absolute	[addr]	Direct memory access
Register indirect	[Rbase]	Load/store through base register
Base + offset	[Rbase + imm]	Small/large displacement
Indexed	[Rbase + Rindex*scale + imm]	For arrays / structures
Stack	PUSH/POP	Special SP-relative addressing

7.8 Encoding Examples

7.8.1 Example 1: Simple Arithmetic

```
ADD R1, R2        ; 16-bit Clownfish mode
```

Binary Encoding:

- No prefix (default mode)
- Opcode: 0x20
- ModR/M: mod=11 (register), reg=R2 (010), r/m=R1 (001)
- Total: 2 bytes

7.8.2 Example 2: Complex Addressing

```
LEAS R4, [R5+R6*4+16]    ; 32-bit Tetra mode
```

Binary Encoding:

- Prefix: 0x40 (Tetra mode indicator)
- Opcode: 0x1E (LEAS)
- ModR/M: mod=10 (32-bit disp), reg=R4 (100), r/m=R5 (101)

- SIB: scale=10 (4x), index=R6 (110), base=R5 (101)
- Displacement: 4 bytes (0x00000010)
- Total: 9 bytes

7.8.3 Example 3: Vector Operation

```
VADD V0, V1, V2          ; 128-bit vector
```

Binary Encoding:

- Prefix: 0x08 (vector selector: 128-bit)
- Opcode: 0x97 (VADD)
- Operands: encoded in following bytes (V0, V1, V2)
- Total: 3–4 bytes

7.8.4 Example 4: Immediate Load

```
MOVI R3, #0xDEADBEEF     ; 32-bit immediate
```

Binary Encoding:

- No prefix (default operand size)
- Opcode: 0x01 (MOVI)
- ModR/M: mod=11, reg=000 (unused), r/m=R3 (011)
- Immediate: 4 bytes (0xDEADBEEF)
- Total: 6 bytes

7.9 Encoding Legend

Notes:

- Rdst, Rsrc denote registers (R0–R31)

mem denotes memory operand

- Prefix byte optional (mode, operand size, vector width)

Column	Meaning
Instr	Mnemonic (with operands)
Class	Instruction class (data/arithmetic/etc.)
Opcode	Primary opcode byte(s)
ModR/M	ModR/M byte usage (if applicable)
SIB	SIB byte usage (if applicable)
Imm/Disp	Immediate/displacement byte size
Micro-Ops	Micro-op breakdown, internal RISC-like ops

8 Instruction Encoding & Micro-Op Mapping

This section provides the complete encoding details and micro-op decomposition for all 160 instructions in the SeaBird ISA.

8.1 Data Movement (24 instructions)

Instruction	Opcode	Encoding	Micro-Operations
MOV Rdst, Rsrc	0x00	R-type	uMOV Rdst,Rsrc
MOVI Rdst, #imm	0x01	I-type (1/2/4/8B)	uMOV Rdst,#imm
MOVZX Rdst, Rsrc	0x02	R-type	uMOV Rdst,Rsrc; uZEXT Rdst
MOVSX Rdst, Rsrc	0x03	R-type	uMOV Rdst,Rsrc; uSEXT Rdst
MOVHI Rdst, Rsrc	0x04	R-type	uMOVHI Rdst,Rsrc
MOVLO Rdst, Rsrc	0x05	R-type	uMOVLO Rdst,Rsrc
MOVSWP Rdst, Rsrc	0x06	R-type	uBSWAP Rdst
LD Rdst, [addr]	0x10	M-type + disp	uLOAD Rdst,[addr]
ST [addr], Rsrc	0x11	M-type + disp	uSTORE [addr],Rsrc
LDI Rdst, #imm	0x12	I-type + disp	uLOAD Rdst,[#addr+imm]
LDB Rdst, [addr]	0x13	M-type (1B)	uLOAD.B Rdst,[addr]
LDH Rdst, [addr]	0x14	M-type (2B)	uLOAD.H Rdst,[addr]
LDW Rdst, [addr]	0x15	M-type (4B)	uLOAD.W Rdst,[addr]
LDQ Rdst, [addr]	0x16	M-type (8B)	uLOAD.Q Rdst,[addr]
STB [addr], Rsrc	0x17	M-type (1B)	uSTORE.B [addr],Rsrc
STH [addr], Rsrc	0x18	M-type (2B)	uSTORE.H [addr],Rsrc
STW [addr], Rsrc	0x19	M-type (4B)	uSTORE.W [addr],Rsrc
STQ [addr], Rsrc	0x1A	M-type (8B)	uSTORE.Q [addr],Rsrc

Instruction	Opcode	Encoding	Micro-Operations
LDP Rdst1, Rdst2, [addr]	0x1B	M-type + pair	uLOAD Rdst1,[addr]; uLOAD Rdst2,[addr+sz]
STP [addr], Rsrc1, Rsrc2	0x1C	M-type + pair	uSTORE [addr],Rsrc1; uSTORE [addr+sz],Rsrc2
LEA Rdst, [addr]	0x1D	M-type	uADDR Rdst,[addr]
LEAS Rdst, [base+idx*s+d]	0x1E	M-type + SIB	uMUL tmp,idx,s; uADD tmp,base,tmp; uADD Rdst,tmp,d
XCHG Rdst, Rsrc	0x1F	R-type	uXCHG Rdst,Rsrc

8.2 Arithmetic (26 instructions)

Instruction	Opcode	Encoding	Micro-Operations
ADD Rdst, Rsrc	0x20	R-type	uADD Rdst,Rsrc
ADDI Rdst, #imm	0x21	I-type (1/2/4/8B)	uADD Rdst,#imm
SUB Rdst, Rsrc	0x22	R-type	uSUB Rdst,Rsrc
SUBI Rdst, #imm	0x23	I-type (1/2/4/8B)	uSUB Rdst,#imm
MUL Rdst, Rsrc	0x24	R-type	uMUL Rdst,Rsrc
MULI Rdst, #imm	0x25	I-type (1/2/4/8B)	uMUL Rdst,#imm
DIV Rdst, Rsrc	0x26	R-type	uDIV Rdst,Rsrc
DIVI Rdst, #imm	0x27	I-type (1/2/4/8B)	uDIV Rdst,#imm
MOD Rdst, Rsrc	0x28	R-type	uMOD Rdst,Rsrc
MODI Rdst, #imm	0x29	I-type (1/2/4/8B)	uMOD Rdst,#imm
NEG Rdst	0x2A	R-type (unary)	uNEG Rdst
INC Rdst	0x2B	R-type (unary)	uINC Rdst
DEC Rdst	0x2C	R-type (unary)	uDEC Rdst
MULH Rdst, Rsrc	0x2D	R-type (high bits)	uMULH Rdst,Rsrc
UMUL Rdst, Rsrc	0x2E	R-type (unsigned)	uUMUL Rdst,Rsrc
UDIV Rdst, Rsrc	0x2F	R-type (unsigned)	uUDIV Rdst,Rsrc
ADDS Rdst, Rsrc	0x30	R-type (saturate)	uADD.SAT Rdst,Rsrc
ADDU Rdst, Rsrc	0x31	R-type (sat uns)	uADD.USAT Rdst,Rsrc

Instruction	Opcode	Encoding	Micro-Operations
SUBS Rdst, Rsrc	0x32	R-type (saturate)	uSUB.SAT Rdst,Rsrc
SUBU Rdst, Rsrc	0x33	R-type (sat uns)	uSUB.USAT Rdst,Rsrc
ABS Rdst, Rsrc	0x34	R-type	uABS Rdst,Rsrc
CLZ Rdst, Rsrc	0x35	R-type	uCLZ Rdst,Rsrc
CTZ Rdst, Rsrc	0x36	R-type	uCTZ Rdst,Rsrc
POPC Rdst, Rsrc	0x37	R-type	uPOPC Rdst,Rsrc

8.3 Logic & Bit Operations (18 instructions)

Instruction	Opcode	Encoding	Micro-Operations
AND Rdst, Rsrc	0x40	R-type	uAND Rdst,Rsrc
OR Rdst, Rsrc	0x41	R-type	uOR Rdst,Rsrc
XOR Rdst, Rsrc	0x42	R-type	uXOR Rdst,Rsrc
NOT Rdst	0x43	R-type (unary)	uNOT Rdst
NAND Rdst, Rsrc	0x44	R-type	uNAND Rdst,Rsrc
NOR Rdst, Rsrc	0x45	R-type	uNOR Rdst,Rsrc
XNOR Rdst, Rsrc	0x46	R-type	uXNOR Rdst,Rsrc
SHL Rdst, Rsrc	0x47	R-type	uSHL Rdst,Rsrc
SHR Rdst, Rsrc	0x48	R-type	uSHR Rdst,Rsrc
SAR Rdst, Rsrc	0x49	R-type	uSAR Rdst,Rsrc
ROL Rdst, Rsrc	0x4A	R-type	uROL Rdst,Rsrc
ROR Rdst, Rsrc	0x4B	R-type	uROR Rdst,Rsrc
BSET Rdst, bit	0x4C	R-type + imm	uBSET Rdst,bit
BCLR Rdst, bit	0x4D	R-type + imm	uBCLR Rdst,bit
BTOG Rdst, bit	0x4E	R-type + imm	uBTOG Rdst,bit
BTST Rdst, bit	0x4F	R-type + imm	uBTST Rdst,bit
MASK Rdst, Rsrc, mask	0x50	R-type + imm	uMASK Rdst,Rsrc,mask
EXT Rdst, Rsrc, range	0x51	R-type + imm	uEXT Rdst,Rsrc,range

8.4 Comparison (10 instructions)

Instruction	Opcode	Encoding	Micro-Operations
CMP Ra, Rb	0x52	R-type	uCMP Ra,Rb (set flags)
CMPI Ra, #imm	0x53	I-type	uCMP Ra,#imm (set flags)
CMPS Ra, Rb	0x54	R-type	uCMP.S Ra,Rb (signed)
CMPU Ra, Rb	0x55	R-type	uCMP.U Ra,Rb (unsigned)
TST Ra, Rb	0x56	R-type	uTST Ra,Rb (bitwise AND, set flags)
TSTI Ra, #imm	0x57	I-type	uTST Ra,#imm (test bits)
MAX Rdst, Rsrc	0x58	R-type	uMAX Rdst,Rsrc (max value)
MIN Rdst, Rsrc	0x59	R-type	uMIN Rdst,Rsrc (min value)
SLT Rdst, Rsrc	0x5A	R-type	uSLT Rdst,Rsrc (set if less than)
SGT Rdst, Rsrc	0x5B	R-type	uSGT Rdst,Rsrc (set if greater)

8.5 Branch / Control Flow (20 instructions)

Instruction	Opcode	Encoding	Micro-Operations
JMP label	0x5C	B-type (rel)	uJMP PC+offset
JMPA addr	0x5D	B-type (abs)	uJMP addr
CALL label	0x5E	B-type (rel)	uPUSH ret_addr; uJMP PC+offset
CALLA addr	0x5F	B-type (abs)	uPUSH ret_addr; uJMP addr
RET	0x60	B-type	uPOP PC
JE label	0x61	B-type (cond)	uBRANCH_IF ZF=1
JNE label	0x62	B-type (cond)	uBRANCH_IF ZF=0
JG label	0x63	B-type (cond)	uBRANCH_IF (SF=OF) & ZF=0
JGE label	0x64	B-type (cond)	uBRANCH_IF SF=OF
JL label	0x65	B-type (cond)	uBRANCH_IF SF!=OF
JLE label	0x66	B-type (cond)	uBRANCH_IF (SF!=OF) — ZF=1
JC label	0x67	B-type (cond)	uBRANCH_IF CF=1
JNC label	0x68	B-type (cond)	uBRANCH_IF CF=0
JO label	0x69	B-type (cond)	uBRANCH_IF OF=1

Instruction	Opcode	Encoding	Micro-Operations
JNO label	0x6A	B-type (cond)	uBRANCH_IF OF=0
JS label	0x6B	B-type (cond)	uBRANCH_IF SF=1
JNS label	0x6C	B-type (cond)	uBRANCH_IF SF=0
JZR reg, label	0x6D	B-type (reg)	uBRANCH_IF reg=0
JNZR reg, label	0x6E	B-type (reg)	uBRANCH_IF reg!=0
BRR reg	0x6F	B-type (ind)	uJMP [reg]
TRAP #imm	0x70	S-type	uTRAP imm (exception)
YIELD	0x71	S-type	uYIELD (scheduler hint)

8.6 Memory / Addressing Extensions (12 instructions)

Instruction	Opcode	Encoding	Micro-Operations
PREFETCH [addr]	0x72	M-type	uPREFETCH [addr] (hint)
FLUSH [addr]	0x73	M-type	uFLUSH [addr] (cache writeback)
INVIC [addr]	0x74	M-type	uINVAL_ICACHE [addr]
INVDC [addr]	0x75	M-type	uINVAL_DCACHE [addr]
LDX Rdst, [addr+]	0x76	M-type	uLOAD Rdst,[addr]; uINC addr
STX [addr+], Rsrc	0x77	M-type	uSTORE [addr],Rsrc; uINC addr
LDN Rdst, [addr-]	0x78	M-type	uDEC addr; uLOAD Rdst,[addr]
STN [addr-], Rsrc	0x79	M-type	uDEC addr; uSTORE [addr],Rsrc
MOVM Rdst, mask	0x7A	X-type	Multi-reg masked move (microcoded)
CPYB dst, src, len	0x7B	X-type	Byte copy loop (microcoded)
CPYW dst, src, len	0x7C	X-type	Word copy loop (microcoded)
MEMFILL dst, val, len	0x7D	X-type	Memory fill loop (microcoded)

8.7 System & Privileged (15 instructions)

Instruction	Opcode	Encoding	Micro-Operations
HLT	0x7E	S-type	uHALT (stop execution)
RESET	0x7F	S-type	uRESET (soft reset)
RDCR Rdst, CRn	0x80	S-type	uREAD_CTRL_REG Rdst,CRn
WRCR CRn, Rsrc	0x81	S-type	uWRITE_CTRL_REG CRn,Rsrc
SYSCALL	0x82	S-type	uSYSCALL (trap to kernel)
SYSRET	0x83	S-type	uSYSRET (return from kernel)
IRET	0x84	S-type	uIRET (return from interrupt)
CLI	0x85	S-type	uCLEAR_IF (disable interrupts)
STI	0x86	S-type	uSET_IF (enable interrupts)
WFI	0x87	S-type	uWAIT_FOR_INTERRUPT
RDTIME Rdst	0x88	S-type	uREAD_TIMER Rdst
RDTS Rdst	0x89	S-type	uREAD_TIMESTAMP Rdst
SLEEP #imm	0x8A	S-type + imm	uSLEEP #imm (power mode)
GETPID Rdst	0x8B	S-type	uGET_PROCESS_ID Rdst
GETTID Rdst	0x8C	S-type	uGET_THREAD_ID Rdst

8.8 Stack & Call Frame (10 instructions)

Instruction	Opcode	Encoding	Micro-Operations
PUSH Rsrc	0x8D	S-type	uDEC SP; uSTORE [SP],Rsrc
POP Rdst	0x8E	S-type	uLOAD Rdst,[SP]; uINC SP
PUSHA	0x8F	X-type	Push all GPRs (R0-R15, microcoded)
POPA	0x90	X-type	Pop all GPRs (R15-R0, microcoded)
ENTER #framesize	0x91	X-type	uPUSH BP; uMOV BP,SP; uSUB SP,size
LEAVE	0x92	X-type	uMOV SP,BP; uPOP BP
PUSHF	0x93	S-type	uDEC SP; uSTORE [SP],FLAGS
POPF	0x94	S-type	uLOAD FLAGS,[SP]; uINC SP
PUSHQ regpair	0x95	S-type	Push register pair (64-bit)
POPQ regpair	0x96	S-type	Pop register pair (64-bit)

8.9 SIMD / Vector (15 instructions)

Instruction	Opcode	Encoding	Micro-Operations
VADD Vd, Va, Vb	0x97	V-type	Per-lane: uADD Vd[i],Va[i],Vb[i]
VSUB Vd, Va, Vb	0x98	V-type	Per-lane: uSUB Vd[i],Va[i],Vb[i]
VMUL Vd, Va, Vb	0x99	V-type	Per-lane: uMUL Vd[i],Va[i],Vb[i]
VDIV Vd, Va, Vb	0x9A	V-type	Per-lane: uDIV Vd[i],Va[i],Vb[i]
VAND Vd, Va, Vb	0x9B	V-type	Per-lane: uAND Vd[i],Va[i],Vb[i]
VOR Vd, Va, Vb	0x9C	V-type	Per-lane: uOR Vd[i],Va[i],Vb[i]
VXOR Vd, Va, Vb	0x9D	V-type	Per-lane: uXOR Vd[i],Va[i],Vb[i]
VSHL Vd, Va, #imm	0x9E	V-type + imm	Per-lane: uSHL Vd[i],Va[i],imm
VSHR Vd, Va, #imm	0x9F	V-type + imm	Per-lane: uSHR Vd[i],Va[i],imm
VDUP Vd, Va	0xA0	V-type	Broadcast: Vd[i]=Va[0] for all i
VLD Vd, [addr]	0xA1	V-type + M	Load vector from memory
VST [addr], Vd	0xA2	V-type + M	Store vector to memory
VABS Vd, Va	0xA3	V-type	Per-lane: uABS Vd[i],Va[i]
VMAX Vd, Va, Vb	0xA4	V-type	Per-lane: uMAX Vd[i],Va[i],Vb[i]
VMIN Vd, Va, Vb	0xA5	V-type	Per-lane: uMIN Vd[i],Va[i],Vb[i]

8.10 Floating Point (10 instructions)

Instruction	Opcode	Encoding	Micro-Operations
FADD Fd, Fa, Fb	0xA6	R-type (FP)	fADD Fd,Fa,Fb (IEEE 754)
FSUB Fd, Fa, Fb	0xA7	R-type (FP)	fSUB Fd,Fa,Fb (IEEE 754)
FMUL Fd, Fa, Fb	0xA8	R-type (FP)	fMUL Fd,Fa,Fb (IEEE 754)
FDIV Fd, Fa, Fb	0xA9	R-type (FP)	fDIV Fd,Fa,Fb (IEEE 754)

Instruction	Opcode	Encoding	Micro-Operations
FSQRT Fd, Fa	0xAA	R-type (FP)	fSQRT Fd,Fa (square root)
FCMP Fa, Fb	0xAB	R-type (FP)	fCMP Fa,Fb (set FP flags)
FCVTI Fd, Ra	0xAC	R-type (conv)	Convert integer Ra to float Fd
FCVTS Ra, Fd	0xAD	R-type (conv)	Convert float Fd to integer Ra
FNEG Fd, Fa	0xAE	R-type (FP)	fNEG Fd,Fa (negate)
FABS Fd, Fa	0xAF	R-type (FP)	fABS Fd,Fa (absolute value)

9 Micro-Operations Architecture

9.1 Hybrid CISC/RISC Design Philosophy

The SeaBird ISA employs a hybrid architecture combining a CISC frontend with a RISC backend, similar to modern processors from Intel and AMD. This design provides:

- **CISC Frontend:** Expressive, compact instructions visible to programmers and compilers
- **RISC Backend:** Simple, uniform micro-operations (µops) executed by the CPU pipeline
- **Performance Benefits:** Combines code density of CISC with execution efficiency of RISC

Execution Flow:

1. Programmer writes CISC-style instructions (e.g., LOAD R1, [R2+R3*4+32])
2. Frontend decoder translates to RISC-like micro-ops
3. Backend execution engine processes micro-ops through standard pipeline stages
4. Micro-ops can be reordered, fused, and optimized independently

This approach enables:

- Out-of-order execution
- Micro-op fusion and optimization

- Superscalar dispatch
- Register renaming at the µop level
- Cache and branch prediction at µop granularity

9.2 Micro-Op Decomposition Rules

The following rules govern how CISC instructions decompose into RISC-like micro-operations:

9.2.1 Rule 1: Single-Operation Instructions → 1 µop

Simple register-to-register operations map directly to single micro-ops.

```
; CISC Instruction -> Micro-Op
ADD R1, R2, R3     -> uADD R1, R2, R3
XOR R4, R5         -> uXOR R4, R5
NEG R3             -> uNEG R3
MOV R6, R7         -> uMOV R6, R7
```

9.2.2 Rule 2: Memory Operations → 2–3 µops

Load/store with simple addressing requires address calculation plus memory access.

```
; CISC Instruction
LOAD R1, [R2+16]

; Micro-Op Breakdown
uADD  Rtmp, R2, #16       ; compute effective address
uLOAD R1, [Rtmp]          ; perform load
```

```
; CISC Instruction
STORE [R3+8], R4

; Micro-Op Breakdown
uADD   Rtmp, R3, #8       ; compute effective address
uSTORE [Rtmp], R4         ; perform store
```

9.2.3 Rule 3: Complex Addressing Modes → 3–5 µops

Complex addressing with base + scaled index + displacement requires multiple steps.

```
; CISC Instruction
LOAD R1, [R2+R3*4+32]

```

```
; Micro-Op Breakdown
uMUL  Rtmp1, R3, #4       ; scale index by 4
uADD  Rtmp2, R2, Rtmp1    ; add base to scaled index
uADD  Rtmp2, Rtmp2, #32   ; add displacement
uLOAD R1, [Rtmp2]         ; perform load
```

9.2.4 Rule 4: Arithmetic with Immediate → 1–2 µops

Immediate arithmetic typically maps to single µop if immediate fits in encoding.

```
; CISC Instruction
ADDI R1, R2, #7

; Micro-Op Breakdown
uADD R1, R2, #7           ; single  op  with embedded immediate
```

For large immediates:

```
; CISC Instruction
ADDI R1, R2, #0xDEADBEEF

; Micro-Op Breakdown
uMOVI Rtmp, #0xDEADBEEF   ; load large immediate
uADD  R1, R2, Rtmp        ; perform addition
```

9.2.5 Rule 5: CISC-Style Compound Operations → 2–6 µops

Complex operations combining multiple actions decompose into sequential µops.

```
; CISC Instruction
PUSH [R3+8]

; Micro-Op Breakdown
uADD   Rtmp1, R3, #8      ; compute source address
uLOAD  Rtmp2, [Rtmp1]     ; load value from memory
uADD   RSP, RSP, #-8      ; adjust stack pointer
uSTORE [RSP], Rtmp2       ; store to stack
```

```
; CISC Instruction
CALL label

; Micro-Op Breakdown
uMOVI  Rtmp1, return_addr ; load return address
uSTORE [RSP-8], Rtmp1     ; push return address
uADD   RSP, RSP, #-8      ; adjust stack pointer
uJMP   label              ; jump to target
```

9.2.6 Rule 6: Branches → 1 µop

Branch instructions map directly to conditional or unconditional µop branches.

```
; CISC Instruction -> Micro-Op
JZ  label          -> uBRANCH_IF_ZERO PC+offset
JMP target         -> uJMP PC+offset
RET                -> uRET (implicit load+jump)
```

9.2.7 Rule 7: Vector Operations → N µops

Vector operations decompose into per-lane µops based on vector width.

```
; CISC Instruction
VADD V0, V1, V2

; Micro-Op Breakdown (for 4-lane vector)
uADD V0.lane0, V1.lane0, V2.lane0
uADD V0.lane1, V1.lane1, V2.lane1
uADD V0.lane2, V1.lane2, V2.lane2
uADD V0.lane3, V1.lane3, V2.lane3
```

Note: Vector µops may execute in parallel on SIMD execution units.

9.2.8 Rule 8: Microcoded Instructions → Variable µops

Special or rarely-used instructions may invoke microcode sequences.

```
; CISC Instruction
PUSHA                     ; push all GPRs

; Micro-Op Breakdown (microcoded)
uSTORE [RSP-8],  R0
uSTORE [RSP-16], R1
uSTORE [RSP-24], R2
...
uSTORE [RSP-248], R31
uADD RSP, RSP, #-256
```

9.3 Micro-Op Execution Properties

9.3.1 Micro-Op Characteristics

All micro-ops share common properties enabling efficient backend execution:

- **Fixed-format:** Uniform µop structure simplifies decode and dispatch

- **Single-operation:** Each µop performs one elementary action
- **Explicit operands:** No implicit side effects (except flags)
- **Register-based:** Temporary registers (T0–T3) used for intermediate values
- **Predictable latency:** Most µops have 1-cycle execution time

9.3.2 Backend Optimization Opportunities

The µop stream enables aggressive backend optimizations:

- **Micro-op fusion:** Adjacent µops combined into single operation
- **Register renaming:** Eliminate false dependencies
- **Out-of-order execution:** Independent µops execute in parallel
- **Speculative execution:** Branch prediction at µop granularity
- **Memory disambiguation:** Reorder loads/stores when safe

9.3.3 Example: Full Instruction Pipeline

```
; Programmer writes:
ADD R1, R2, [R3+R4*8+64]

; Frontend decodes to micro-ops:
uMUL  T0, R4, #8          ; cycle 1: scale index
uADD  T1, R3, T0          ; cycle 2: add base
uADD  T1, T1, #64         ; cycle 3: add displacement
uLOAD T2, [T1]            ; cycle 4-6: memory access
uADD  R1, R2, T2          ; cycle 7: final addition

; Backend may optimize to:
; - Fuse address calculation  ops
; - Execute uMUL and other ops out-of-order
; - Rename T0, T1, T2 to physical registers
; - Overlap memory access with other work
```

10 Instruction Set Reference

The SeaBird ISA defines 160 instructions across 10 functional categories. Each instruction may decompose into one or more micro-ops during execution.

10.1 Data Movement (24 instructions)

10.1.1 Basic Moves

```
MOV     dst, src          ; register or memory move
MOVI    dst, imm          ; move immediate
MOVZX   dst, src          ; zero-extend
MOVSX   dst, src          ; sign-extend
MOVHI   dst, src          ; move high half
MOVLO   dst, src          ; move low half
MOVSWP  dst, src          ; byte-swap endian
```

10.1.2 Memory Access

```
LD      dst, [addr]       ; load
ST      [addr], src       ; store
LDI     dst, imm(addr)    ; load from immediate address
LDB     dst, [addr]       ; load byte
LDH     dst, [addr]       ; load halfword
LDW     dst, [addr]       ; load word
LDQ     dst, [addr]       ; load quadword
STB     [addr], src       ; store byte
STH     [addr], src       ; store halfword
STW     [addr], src       ; store word
STQ     [addr], src       ; store quadword
```

10.1.3 Special Variants

```
LDP     dst1, dst2, [addr]      ; load pair
STP     [addr], src1, src2      ; store pair
LEA     dst, [addr]             ; compute effective address
LEAS    dst, [base + index*scale + imm]
XCHG    dst, src                ; atomic swap
```

10.2 Arithmetic (26 instructions)

10.2.1 Integer Arithmetic

```
ADD     r, v              ; add
ADDI    r, imm            ; add immediate
SUB     r, v              ; subtract
SUBI    r, imm            ; subtract immediate
```

```
MUL     dst, src          ; multiply
MULI    dst, imm          ; multiply immediate
DIV     dst, src          ; divide
DIVI    dst, imm          ; divide immediate
MOD     dst, src          ; modulo
MODI    dst, imm          ; modulo immediate
NEG     r                 ; negate
INC     r                 ; increment
DEC     r                 ; decrement
```

10.2.2 Extended / 64-bit

```
MULH    dst, src          ; high-half multiply
UMUL    dst, src          ; unsigned multiply
UDIV    dst, src          ; unsigned divide
```

10.2.3 Saturating Arithmetic

```
ADDS    dst, src          ; signed saturating add
ADDU    dst, src          ; unsigned saturating add
SUBS    dst, src          ; signed saturating subtract
SUBU    dst, src          ; unsigned saturating subtract
```

10.2.4 Bit Arithmetic

```
ABS     dst, src          ; absolute value
CLZ     dst, src          ; count leading zeros
CTZ     dst, src          ; count trailing zeros
POPC    dst, src          ; popcount
```

10.3 Logic & Bit Operations (18 instructions)

```
AND     r, v              ; logical AND
OR      r, v              ; logical OR
XOR     r, v              ; logical XOR
NOT     r                 ; logical NOT
NAND    r, v              ; NAND
NOR     r, v              ; NOR
XNOR    r, v              ; XNOR
SHL     r, v              ; shift left
SHR     r, v              ; shift right
```

```
SAR     r, v              ; shift arithmetic right
ROL     r, v              ; rotate left
ROR     r, v              ; rotate right
BSET    r, bit            ; bit set
BCLR    r, bit            ; bit clear
BTOG    r, bit            ; bit toggle
BTST    dst, bit          ; test bit
MASK    dst, src, mask    ; apply mask
EXT     dst, src, bitrange ; extract bit range
```

10.4 Comparison (10 instructions)

```
CMP     a, b              ; compare (sets flags)
CMPI    a, imm            ; compare immediate
CMPS    a, b              ; signed compare
CMPU    a, b              ; unsigned compare
TST     a, b              ; logical test
TSTI    a, imm            ; test immediate
MAX     dst, src          ; maximum
MIN     dst, src          ; minimum
SLT     dst, src          ; set if less than
SGT     dst, src          ; set if greater than
```

10.5 Branching & Control Flow (20 instructions)

10.5.1 Unconditional

```
JMP     lbl               ; jump to label
JMPA    addr              ; jump to absolute address
CALL    lbl               ; call subroutine
CALLA   addr              ; call absolute address
RET                       ; return from subroutine
```

10.5.2 Conditional (flag-based)

```
JE      lbl               ; jump if equal
JNE     lbl               ; jump if not equal
JG      lbl               ; jump if greater
JGE     lbl               ; jump if greater or equal
JL      lbl               ; jump if less
JLE     lbl               ; jump if less or equal
JC      lbl               ; jump if carry
```

```
JNC     lbl                 ; jump if not carry
JO      lbl                 ; jump if overflow
JNO     lbl                 ; jump if not overflow
JS      lbl                 ; jump if sign
JNS     lbl                 ; jump if not sign
```

10.5.3 Register-based

```
JZR     reg, lbl            ; jump if register zero
JNZR    reg, lbl            ; jump if register not zero
BRR     reg                 ; jump via register
```

10.5.4 Special

```
TRAP    imm                 ; software trap
YIELD                       ; yield execution
```

10.6 Memory / Addressing Extensions (12 instructions)

```
PREFETCH    [addr]          ; prefetch cache line
FLUSH       [addr]          ; flush cache line
INVIC       [addr]          ; invalidate icache line
INVDC       [addr]          ; invalidate dcache line
LDX         dst, [addr+]    ; load and post-increment
STX         [addr+], src    ; store and post-increment
LDN         dst, [addr-]    ; load and post-decrement
STN         [addr-], src    ; store and post-decrement
MOVM        dst, mask       ; masked move
CPYB        dst, src, len   ; copy bytes
CPYW        dst, src, len   ; copy words
MEMFILL     dst, value, len ; fill memory
```

10.7 System & Privileged (15 instructions)

```
HLT                         ; halt processor
RESET                       ; system reset
RDCR    dst, crN            ; read control register
WRCR    crN, src            ; write control register
SYSCALL                     ; system call
SYSRET                      ; return from syscall
IRET                        ; interrupt return
```

```
CLI                         ; clear interrupts
STI                         ; set interrupts
WFI                         ; wait for interrupt
RDTIME  dst                 ; read time
RDTS    dst                 ; read timestamp
SLEEP   imm                 ; sleep for duration
GETPID                      ; get process ID
GETTID                      ; get thread ID
```

10.8 Stack & Call Frame (10 instructions)

```
PUSH    r                   ; push register
POP     r                   ; pop register
PUSHA                       ; push all GPRs
POPA                        ; pop all GPRs
ENTER   imm                 ; create stack frame
LEAVE                       ; destroy stack frame
PUSHF                       ; push flags
POPF                        ; pop flags
PUSHQ   regpair             ; push register pair
POPQ    regpair             ; pop register pair
```

10.9 SIMD / Vector (15 instructions)

Assuming 128-bit or 256-bit V registers:

```
VADD    vd, va, vb          ; vector add
VSUB    vd, va, vb          ; vector subtract
VMUL    vd, va, vb          ; vector multiply
VDIV    vd, va, vb          ; vector divide
VAND    vd, va, vb          ; vector AND
VOR     vd, va, vb          ; vector OR
VXOR    vd, va, vb          ; vector XOR
VSHL    vd, va, imm         ; vector shift left
VSHR    vd, va, imm         ; vector shift right
VDUP    vd, va              ; vector duplicate
VLD     vd, [addr]          ; vector load
VST     [addr], vd          ; vector store
VABS    vd, va              ; vector absolute value
VMAX    vd, va, vb          ; vector maximum
VMIN    vd, va, vb          ; vector minimum
```

10.10 Floating Point (10 instructions)

```
FADD    fd, fa, fb        ; floating-point add
FSUB    fd, fa, fb        ; floating-point subtract
FMUL    fd, fa, fb        ; floating-point multiply
FDIV    fd, fa, fb        ; floating-point divide
FSQRT   fd, fa            ; floating-point square root
FCMP    fa, fb            ; floating-point compare
FCVTI   fd, ra            ; convert int to float
FCVTS   ra, fd            ; convert float to int
FNEG    fd                ; floating-point negate
FABS    fd                ; floating-point absolute value
```

Total: 160 instructions

11 Assembler Directives

This section documents all assembler directives supported by the toolchain. Directives control code organization, symbol visibility, data layout, optimization behavior, and architecture-specific features.

11.1 Code & Data Layout Directives

`.text` Begin a code section. Assembler switches to instruction-emission mode.

`.data` Begin a data section. Used for static variables, lookup tables, and buffers.

`.bss` Begin an uninitialized data section. Reserves space without emitting bytes.

`.align` *n* Align the next emitted object to an n-byte boundary. Particularly beneficial for aligning instructions or micro-tables in CISC frontend code.

`.org` *addr* Set the absolute location counter to *addr*. Primarily used for firmware, vector tables, and boot ROM.

11.2 Symbol & Label Directives

`.global` *symbol* Export a symbol for use by other modules.

`.extern` *symbol* Declare a symbol to be resolved by the linker.

`.equ` *name, value* Create a constant. Equivalent to `#define` in C.

`.set` *name, value* Same as `.equ`, but may be reassigned later.

`.macro` *name args...* Begin a macro definition.

.endm End a macro definition.

11.3 Data Definition Directives

.byte *b1, b2, ...* Emit raw 8-bit bytes.

.word *w1, w2, ...* Emit 16-bit values.

.dword *d1, d2, ...* Emit 32-bit values.

.qword *q1, q2, ...* Emit 64-bit values.

.float *f1, f2, ...* Emit 32-bit IEEE floating-point values.

.double *d1, d2, ...* Emit 64-bit IEEE floating-point values.

.ascii "*text*" Emit raw ASCII bytes without null terminator.

.asciz "*text*" / **.string "*text*"** Emit a null-terminated string.

.space *n* Reserve n bytes of space (zero-initialized or uninitialized).

11.4 Control Directives

.include "*file*" Insert another source file at this location.

.ifdef *SYM* / **.ifndef *SYM*** Begin conditional assembly block.

.endif End conditional assembly block.

11.5 Architecture, Mode & Optimization Control

These directives are critical for controlling behavior in the hybrid ISA environment.

.cpu *model* Declare target CPU model or extension level. Enables assembler warnings if instructions are not supported on the specified model.

.arch *ext1, ext2, ...* Enable optional ISA extensions (e.g., `VECTOR`, `SYS`, `CRYPTO`).

.mopt *level* Set assembler micro-op optimization level:

- `0` – No micro-fusion (debug-friendly)
- `1` – Allow simple micro-op fusion
- `2` – Full aggressive micro-crunching mode

.mode *MODE_NAME* Switch code-generation mode. Supported modes include:

- `CLOWNFISH` – 16-bit legacy mode
- `TETRA` – 32-bit base mode
- `DRAGONET` – 64-bit Meisei mode
- `DROPLET` – 128-bit hyper mode
- `REALTIME` – Disallow slow CISC expansions; enforce micro-op minimality

`.uops on/off` Show generated micro-ops in listing output.

11.6 Debugging, Listing & Metadata

`.file "`*`name`*`"` Set debug filename for debugging symbols.

`.line `*`n`* Force encoding of source line number n for debugging.

`.section `*`name`* Create or switch to user-defined sections. Useful for custom loaders or non-standard binary formats.

`.message "`*`text`*`"` Print informational message during assembly.

`.warning "`*`text`*`"` Print warning message (assembly continues).

`.error "`*`text`*`"` Emit fatal error and halt assembly.

11.7 Special Optimization Directives

These directives provide fine-grained control over assembler optimization behavior.

`.hint microburst` Mark the following block as performance-critical. Instructs the optimizer to apply aggressive optimizations.

`.prefetch `*`addr`* Emit an embedded prefetch hint micro-op for address *addr*.

`.autovec on/off` Enable or disable automatic loop vectorization by the assembler.

`.trustme` Disable **all** safety checks. Assembler will accept code without validation. *Use with extreme caution.*

`.sleepdeprived` Convenience directive equivalent to:

```
.mopt 2
.uops on
.autovec on
```

Enables maximum optimization and diagnostic output. Named for late-night coding sessions.

12 Example Instructions With Micro-Op Breakdown

Instruction	Micro-Ops
ADD R1,R2,R3	uADD R1,R2,R3
LOAD R1,[R2+R3*4+32]	uMUL Rtmp,R3,#4; uADD Rtmp,R2,Rtmp; uADD Rtmp,Rtmp,#32; uLOAD R1,[Rtmp]
CALL label	uMOVI TMP1,return_addr; uSTORE [SP-8],TMP1; uALU SP,SP,-8,ADD; uJMP label
RET	uLOAD TMP1,[SP]; uALU SP,SP,8,ADD; uJMP TMP1
VADD V0,V1,V2	loop over lanes: uADD lane0,lane1,lane2

13 Instruction Latency & Performance Characteristics

13.1 Typical Instruction Latencies

Performance characteristics for a reference implementation with out-of-order execution:

Operation Type	Latency	Throughput
Single µop (ADD, MOV, XOR)	1 cycle	4/cycle
Memory load (L1 hit)	4 cycles	2/cycle
Memory store (L1 hit)	1 cycle	2/cycle
Integer multiply	3 cycles	1/cycle
Integer divide	10–20 cycles	1/10 cycles
Floating-point add/sub	3 cycles	2/cycle
Floating-point multiply	4 cycles	2/cycle
Floating-point divide	10–15 cycles	1/10 cycles
Vector ops (per lane)	1–4 cycles	varies
Microcoded instructions	3–20 cycles	varies
Branch (predicted)	1 cycle	2/cycle
Branch (mispredicted)	15–20 cycles	penalty

13.2 Cache and Memory Hierarchy

Typical implementation memory subsystem:

Level	Size	Latency	Bandwidth
L1 Data Cache	32–64 KB	4 cycles	64 GB/s
L1 Instruction Cache	32–64 KB	4 cycles	64 GB/s
L2 Unified Cache	256–512 KB	12 cycles	32 GB/s
L3 Unified Cache	4–16 MB	40 cycles	16 GB/s
Main Memory (DDR4)	8–64 GB	100–300 cycles	25 GB/s

13.3 Pipeline Characteristics

- **Pipeline depth:** 14–20 stages (typical superscalar implementation)
- **Fetch width:** 4–6 instructions per cycle
- **Decode width:** 4–6 instructions → 8–12 µops per cycle
- **Execution width:** 8–10 µops per cycle
- **Retire width:** 4–6 instructions per cycle
- **ROB size:** 192–256 µops (reorder buffer)
- **Branch prediction:** Two-level adaptive with 4K–8K entry BTB

14 FLAGS Register

14.1 Status Flags

The FLAGS register contains condition codes and processor state. Bits are arranged as follows:

Bit	Name	Symbol	Description
0	Carry	CF	Carry out of MSB or borrow
1	Reserved	–	Must be 0
2	Parity	PF	Even parity of low byte
3	Reserved	–	Must be 0
4	Auxiliary Carry	AF	Carry from bit 3 to bit 4
5	Reserved	–	Must be 0
6	Zero	ZF	Result is zero
7	Sign	SF	Sign bit of result (MSB)
8	Trap	TF	Single-step debugging
9	Interrupt Enable	IF	Interrupts enabled
10	Direction	DF	String operation direction
11	Overflow	OF	Signed arithmetic overflow

Bit	Name	Symbol	Description
12–13	I/O Privilege Level	IOPL	I/O privilege (0–3)
14	Nested Task	NT	Nested task flag
15	Reserved	–	Must be 0
16	Resume	RF	Resume flag
17	Virtual 8086	VM	Virtual 8086 mode
18	Alignment Check	AC	Alignment check enabled
19	Virtual Interrupt	VIF	Virtual interrupt flag
20	Virtual Int Pending	VIP	Virtual interrupt pending
21	ID	ID	CPUID support
22–31	Reserved	–	Must be 0

14.2 Flag Modification

Instructions affect flags as follows:

- **Arithmetic:** ADD, SUB, MUL, DIV modify CF, ZF, SF, OF, PF, AF
- **Logic:** AND, OR, XOR clear CF and OF, modify ZF, SF, PF
- **Shift/Rotate:** Modify CF and OF
- **Compare:** CMP, TEST modify all arithmetic flags
- **Branches:** Read flags but do not modify

15 Exception & Interrupt Handling

15.1 Exception Vector Table

The exception vector table begins at address 0xCAFE and contains handlers for system events:

Vector	Mnemonic	Type	Description
0x00	RESET	System	Power-on/reset
0x01	DIV_ZERO	Fault	Division by zero
0x02	DEBUG	Trap	Debug/single-step
0x03	NMI	Interrupt	Non-maskable interrupt
0x04	BREAKPOINT	Trap	Software breakpoint
0x05	OVERFLOW	Trap	Arithmetic overflow
0x06	BOUND	Fault	Bounds check failure
0x07	INVALID_OP	Fault	Invalid opcode
0x08	NO_FPU	Fault	FPU not available
0x09	DOUBLE_FAULT	Abort	Double fault

Vector	Mnemonic	Type	Description
0x0A	INVALID_TSS	Fault	Invalid task segment
0x0B	SEG_NOT_PRESENT	Fault	Segment not present
0x0C	STACK_FAULT	Fault	Stack segment fault
0x0D	GPF	Fault	General protection fault
0x0E	PAGE_FAULT	Fault	Page fault
0x0F	RESERVED	–	Reserved
0x10	FPU_ERROR	Fault	Floating-point error
0x11	ALIGN_CHECK	Fault	Alignment check
0x12	MACHINE_CHECK	Abort	Machine check
0x13	SIMD_ERROR	Fault	SIMD floating-point
0x14–0x1F	RESERVED	–	Reserved for future use
0x20–0xFF	USER_INT	Interrupt	External/software interrupts

15.2 Exception Handling Mechanism

When an exception occurs:

1. Current instruction address saved to CFAR (CISC Frontend Address Register)
2. Current FLAGS saved to stack
3. Current IP saved to stack
4. IF flag cleared (interrupts disabled)
5. Control transferred to exception handler via vector table
6. Handler executes, performs recovery or termination
7. Handler returns via IRET instruction

15.3 Privilege Level Transitions

Exceptions may cause privilege level changes:

- **User → Kernel:** On system call, page fault, or protection violation
- **Kernel → Kernel:** On nested exceptions
- **Kernel → User:** On IRET or SYSRET

Stack switching occurs automatically when privilege level changes, using kernel stack pointer from special register.

16 Control Registers

16.1 System Control Registers

Control registers (CR0–CR15) manage system-level functionality:

Register	Name	Purpose
CR0	System Control	Protected mode, paging, FPU control
CR1	Reserved	Reserved for future use
CR2	Page Fault Linear Address	Faulting address on page fault
CR3	Page Directory Base	Physical address of page directory
CR4	Extended Control	Additional CPU features
CR5–CR7	Reserved	Reserved for future use
CR8	Task Priority	Interrupt priority level
CR9–CR15	Implementation-specific	Vendor-specific features

16.2 CR0 Bit Definitions

Bit	Name	Description
0	PE	Protected Mode Enable
1	MP	Monitor Coprocessor
2	EM	Emulation (FPU emulation)
3	TS	Task Switched
4	ET	Extension Type
5	NE	Numeric Error
16	WP	Write Protect
18	AM	Alignment Mask
29	NW	Not Write-through
30	CD	Cache Disable
31	PG	Paging Enable

17 Memory Management

17.1 Paging Mechanism

The SeaBird ISA supports hierarchical paging in Tetra and Dragonet modes:

Tetra Mode (32-bit):

- Two-level page tables

- Page size: 4 KB (standard) or 4 MB (large pages)
- Virtual address space: 4 GB
- Page directory: 1024 entries
- Page table: 1024 entries per directory

Dragonet Mode (64-bit):

- Four-level page tables (PML4)
- Page size: 4 KB, 2 MB (huge), or 1 GB (gigantic)
- Virtual address space: 256 TB (48-bit addresses)
- Canonical address form required

17.2 Page Table Entry Format

Bits	Field	Description
0	P	Present (page is in memory)
1	R/W	Read/Write (0=read-only, 1=writable)
2	U/S	User/Supervisor (0=kernel, 1=user)
3	PWT	Page Write-Through
4	PCD	Page Cache Disable
5	A	Accessed (set by CPU)
6	D	Dirty (written to)
7	PS	Page Size (0=4KB, 1=large)
8	G	Global page
9–11	AVL	Available for OS use
12–51	PFN	Physical Frame Number
52–62	AVL	Available for OS use
63	XD	Execute Disable (NX bit)

17.3 Translation Lookaside Buffer (TLB)

- **Data TLB:** 64–128 entries for load/store translations
- **Instruction TLB:** 64–128 entries for instruction fetch
- **Unified TLB (L2):** 512–1024 entries

- **Huge page support:** Separate entries for 2MB/1GB pages
- **PCID support:** Process Context ID for tagged TLB entries

18 Programming Examples

18.1 Example 1: Simple Loop

```
; Calculate sum of array elements
; R1 = array pointer, R2 = count, R3 = sum

        MOVI    R3, #0              ; sum = 0
loop:
        LD      R4, [R1]            ; load array[i]
        ADD     R3, R3, R4          ; sum += array[i]
        ADDI    R1, #4              ; i++
        SUBI    R2, #1              ; count--
        JNZ     loop                ; if (count != 0) goto loop

        ; Result in R3
```

18.2 Example 2: Function Call with Stack Frame

```
; Function: int add(int a, int b)
; Arguments: R0 = a, R1 = b
; Returns: R0 = result

add_function:
        ENTER   #16                 ; create stack frame (16 bytes)
        PUSH    R6                  ; save callee-saved register
        PUSH    R7

        MOV     R6, R0              ; save parameter a
        MOV     R7, R1              ; save parameter b
        ADD     R0, R6, R7          ; result = a + b

        POP     R7                  ; restore registers
        POP     R6
        LEAVE                       ; destroy stack frame
        RET                         ; return to caller

; Caller:
main:
```

```
        MOVI    R0, #10             ; first argument
        MOVI    R1, #20             ; second argument
        CALL    add_function        ; call function
        ; result in R0 (= 30)
```

18.3 Example 3: Vector Operations

```
; Vector addition: C = A + B (4 elements each)
; V0 = vector A, V1 = vector B, V2 = result C

        VLD     V0, [R1]            ; load vector A
        VLD     V1, [R2]            ; load vector B
        VADD    V2, V0, V1          ; C = A + B
        VST     [R3], V2            ; store result
```

18.4 Example 4: System Call

```
; System call: write(fd, buffer, count)
; Syscall number in R0, args in R1, R2, R3

        MOVI    R0, #1              ; syscall: write
        MOVI    R1, #1              ; fd = stdout
        LEA     R2, [message]       ; buffer pointer
        MOVI    R3, #13             ; count = 13 bytes
        SYSCALL                     ; invoke kernel
        ; return value in R0

message:
        .ascii  "Hello, World!"
```

19 Implementation Notes

19.1 Design Considerations for Implementers

19.1.1 Frontend Decoder

- Complex instructions should decode to µop sequences in 1–2 cycles
- Maintain µop cache for frequently-decoded instruction sequences
- Support macro-op fusion (e.g., CMP + JZ → single µop)

19.1.2 Execution Backend

- Minimum 4-wide superscalar for competitive performance
- Out-of-order execution with register renaming
- Unified reservation station or distributed scheduler
- Memory disambiguation for load/store reordering

19.1.3 Branch Prediction

- Two-level adaptive predictor recommended
- BTB (Branch Target Buffer) with 4K–8K entries
- Return stack buffer with 16–32 entries
- Indirect branch predictor for virtual calls

19.2 Compliance Levels

Implementations may target different compliance levels:

- **Level 0 (Minimal):** Clownfish mode only, single-issue, in-order
- **Level 1 (Basic):** Tetra mode, 2-way superscalar, simple branch prediction
- **Level 2 (Standard):** Dragonet mode, 4-way superscalar, out-of-order
- **Level 3 (High-Performance):** Full vector support, 6+ way, aggressive µop fusion

20 Revision History

Version	Date	Author	Changes
1.0	2025-12-07	SeaBird Team	Initial release
1.1	2026-01-28	Miyamii	Updated Directives

21 Acknowledgments

The SeaBird ISA draws inspiration from several influential architectures:

- x86-64 (Intel/AMD) - Binary encoding format, ModR/M/SIB structure
- ARM - Register naming conventions, clean instruction orthogonality
- RISC-V - Modern ISA design principles, extensibility
- IBM POWER - Hybrid CISC/RISC execution model

22 Glossary

CISC Complex Instruction Set Computer - Architecture with variable-length, multi-operation instructions

RISC Reduced Instruction Set Computer - Architecture with fixed-length, single-operation instructions

µop (Micro-op) Micro-operation - Internal RISC-like operation into which CISC instructions decompose

GPR General-Purpose Register - Registers available for general computation (R0–R31)

MMU Memory Management Unit - Hardware managing virtual-to-physical address translation

TLB Translation Lookaside Buffer - Cache for page table entries

ROB Reorder Buffer - Structure for out-of-order execution commit

BTB Branch Target Buffer - Cache for branch target addresses

SIMD Single Instruction Multiple Data - Parallel processing of vector data

ModR/M Mode-Register-Memory byte - Encoding byte specifying operands

SIB Scale-Index-Base byte - Encoding byte for complex addressing modes